

Lecture 2: Binary Search

Lecturer: Jasper Lee

Scribe: Jasper Lee

In Lecture 1, we established the basic formulation of binary search over a sorted array. We saw that the core of proving the correctness of a binary search implementation lies in properly defining the loop invariant: bounding the search space such that the element we are looking for (if it exists) resides strictly within the active indices $[L, R]$.

Today, we will see that binary search is in fact a powerful tool, and a much more general paradigm than simply searching for values in a sorted array.

1 Searching in a Rotated Sorted Array

Consider the problem of finding the minimum element in a “rotated” sorted array.

Setting: Let A be a sorted array of n **distinct** values. Suppose we are given A' , which is a cyclically shifted (rotated) version of A . Our goal is to find the index t corresponding to the original location of $A[0]$ (the minimum element of the array).

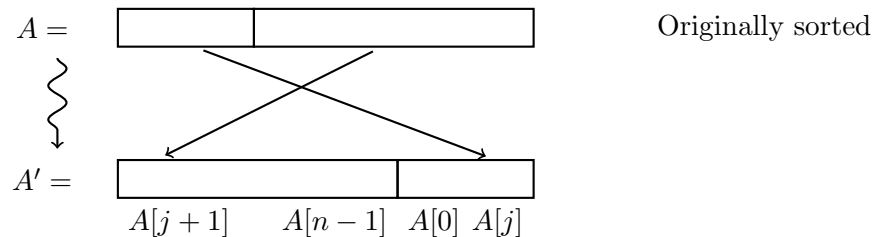


Figure 1: A rotated sorted array A' is formed by cyclically shifting blocks of the original sorted array A .

We can solve this problem using a modified binary search.

Note on Implementation Details: For written homeworks and exams in this course, do not worry about off-by-one errors (e.g. whether a bound should be m or $m - 1$). You only need to worry about exact boundary arithmetic for coding assignments.

The core idea is to maintain a loop invariant and compare the midpoint to the endpoints to determine which half of the array contains t (location of the minimum array value, as seen in Figure 1).

Loop invariant: Throughout the execution of the loop, we always have $t \in [L, R]$. Consequently, the subarray $A'[L..R]$ is always a valid rotated sorted array containing the minimum element.

To update our bounds, we compare the midpoint element $A'[m]$ with the rightmost active element $A'[R]$ (we can also compare with $A'[L]$ instead for a symmetric but technically different algorithm). Because the elements are distinct, we have two cases:

1. **If** $A'[m] > A'[R]$: t must occur strictly after m , meaning the minimum element is in the right half. We update the bound to $L \leftarrow m$.
2. **If** $A'[m] < A'[R]$: The right half from m to R is strictly increasing and does not contain t , so t is either m or to its left. We update the bound to $R \leftarrow m$.

1.1 Challenge Problem: What about duplicates?

A natural follow-up question is: What happens if we allow item duplicates in the rotated array?

It turns out the problem becomes significantly harder, requiring $\Omega(n)$ time in the worst case. We will discuss the intuition here, but not give a fully formal proof.

Consider an adversary who constructs an array of length n consisting entirely of 1s, except for a single 0. The target minimum is the 0, and by rotation can be anywhere in the array.

It can be shown that any deterministic algorithm must make at least $n - 1$ queries in the worst case to guarantee finding the unique 0 in an array of 1s.

2 Single in Pairs

Let us now see another application of binary search outside of searching in a sorted array.

Setting: We are given a sorted array A of n items, where n is an odd integer. Exactly one element appears uniquely in the array. All other $n - 1$ elements appear exactly twice (in adjacent pairs). Our goal is to find the lone unique element.

Idea: For midpoint m , query $A[m - 1]$, $A[m]$, and $A[m + 1]$. Based on these queries, we can determine whether the unique element lies to the left or right of our current position by using a parity argument (as detailed in Figure 2).

3 possibilities

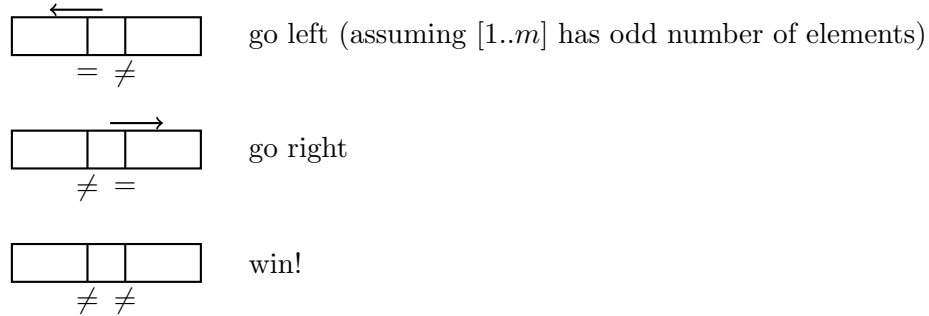


Figure 2: Visualizing the three search possibilities based on local equality checks.

Runtime and Correctness: The runtime is clearly $O(\log n)$ since we halve the search space based on constant-time parity checks.

Loop Invariant: $A[L..R]$ is an array containing the unique element, and all other elements within this subarray appear in pairs.

3 Binary Search: Connecting Optimization to Decision

The previous examples look basic and somewhat silly, but binary search is actually very powerful.

In particular, binary search connects (monotone) optimization problems to (monotone) decision problems, as we explain in this section.

Specifically, consider:

- **Problem 1 (Optimization):** Find the maximum integer k such that a property $P(k)$ is true. Here, we require $P(k)$ to be *monotonic*: if $P(k)$ is true for some k , it implies $P(k')$ for all smaller k' .
- **Problem 2 (Decision):** Given a fixed integer k' , output whether $P(k')$ is true or false.

These two problems turn out to be *equivalent* in a sense, that solving one implies we can solve the other.

The easy direction is Problem 1 $\rightarrow$ Problem 2: to solve Problem 2, just solve Problem 1 for the maximum k and compare with k' .

The more interesting direction we consider now: to solve Problem 1, use binary search over the function (or implicit array) $A[k] = \mathbb{1}\{P(k)\}$ over its possible domain.

3.1 Example: The Stick Cutting Problem

Setting: We are given a list of n sticks with integer lengths $L_1, L_2, \dots, L_n$, and a target integer m . We can cut any stick of length ℓ into two sticks of integer lengths ℓ_a and ℓ_b such that $\ell_a + \ell_b = \ell$. These new sticks can be further divided. Our goal is to find the maximum integer length k exactly such that we can produce at least m sticks, and it is ok if there are other leftover sticks of various lengths.

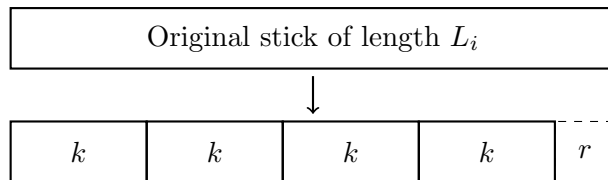


Figure 3: A stick of length L_i can be cut into pieces of length k , leaving some remainder r .

The above is an optimization problem, so what is the corresponding decision problem? It is:

$$P(k) = \{\text{We can produce } \geq m \text{ sticks of length exactly } k\}$$

We have to show that $P(k)$ is monotonic. Why does $P(k) \Rightarrow P(k')$ for $k \geq k'$? If $P(k)$ is true, we have successfully formed m sticks of length k . To form m sticks of length $k' < k$, we can simply trim off a piece of length $k - k'$ from each of the m sticks. Thus, the property holds for all smaller lengths.

Next, how do we check $P(k)$ for a fixed k ? For any original stick of length L_i , we can produce $\lfloor L_i/k \rfloor$ many sticks of length k . Therefore, to check $P(k)$, we compute:

$$\sum_{i=1}^n \left\lfloor \frac{L_i}{k} \right\rfloor$$

and return true if this sum is $\geq m$, and false otherwise. This check takes $O(n)$ time.

To solve the overall optimization problem, we binary search over the possible values of k . The domain of valid values of k is bounded between 1 and $L = \max(L_i)$.

Note: For homeworks and exams, once you establish a binary search over a monotonic array or property, there is no need to re-analyze the binary search itself.

Runtime Analysis: The search space has size L , so the binary search requires $O(\log L)$ steps. In each step, we invoke our decision checker, which takes $O(n)$ time. The total runtime is therefore at most $O(n \log L)$.